

Lecture 4 - Collaborative Filtering

DS 4420: Machine Learning and Data Mining 2

Eric Gerber

`e.gerber@northeastern.edu`

Northeastern University, Boston, USA



Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Updates

- Homework 2 due tonight **Monday, Feb. 2 by 11:59 pm**
 - Homework 3 will be posted then, due **Friday, Feb. 13** (covers this material)
- Quiz 3 **currently** planned for this **Thursday, Feb. 5**
 - Depending on how quickly I get through these lectures, we may push it back to **Monday, Feb. 9**, and I will update the Tentative Schedule on Canvas accordingly
- Our Final Exam date/time has been set for **Wednesday, April 22 at 10:30 am**
 - Please fill out an exam conflict form as soon as possible if you have another exam at that time.

Topics for Today:

- 1 Collaborative Filtering
 - Similarity Scores and Making Recommendations
 - Content-based/Hybrid Filtering
 - User-user vs. item-item CF
 - (if time) SVD-Based Filtering

Push-Up Counter:

- Section 1: 3
- Section 4: 4

Today

- 1 Similarity**
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Norms and Similarity Scores

In CS 2810/DS 3000, and DS 4400, we've learned a few different ways of measuring the **similarity** between vectors (and thus data points), especially

- L1-norm (Manhattan distance; the sum of the absolute values of the difference between two vectors)
- L2-norm (Euclidean distance; the square root of the squared differences)
- L_∞ -norm (Chebyshev distance; the maximum of the absolute differences)
- Cosine Similarity (the cosine of the angle between two vectors)

In this lecture, we'll focus on the basics of **collaborative filtering**, an approach which uses distance measures to identify observations/users with similar traits. It is most often used as a basis for **recommender systems**.

Norms and Similarity Scores

Imagine we have three songs from our class playlist and their artist popularity and duration in seconds as vectors of information (before, and after scaling).

- double take, by Dhruv: $x = \begin{bmatrix} 66 \\ 171.74 \end{bmatrix} \Rightarrow \begin{bmatrix} -0.50 \\ -0.73 \end{bmatrix}$
- One Last Kiss, by Hikaru Utada: $y = \begin{bmatrix} 73 \\ 252.03 \end{bmatrix} \Rightarrow \begin{bmatrix} -0.08 \\ 0.05 \end{bmatrix}$
- Mr. Brightside, by The Killers: $z = \begin{bmatrix} 80 \\ 222.97 \end{bmatrix} \Rightarrow \begin{bmatrix} 0.34 \\ -0.23 \end{bmatrix}$

The third song is Dr. Gerber's favorite song. Using the (admittedly limited) data that you have access to, which of the other two songs would you recommend to Dr. Gerber? Which one (x or y) is most similar to z ?

Basic Ideas

Consider calculating the L_1 , L_2 , L_∞ , and Cosine similarity scores (see **collab_filter.ipynb** and **collab_filter.R**) between the scaled z and each of x and y , what would they tell us?

L1	L2	L_∞	Cosine
$\ z - x\ _1 = 1.33$	$\ z - x\ _2 = 0.98$	$\ z - x\ _\infty = 0.84$	$S_C(z, x) = -0.001$
$\ z - y\ _1 = 0.70$	$\ z - y\ _2 = 0.51$	$\ z - y\ _\infty = 0.42$	$S_C(z, y) = -0.996$

The norms tell us that Mr. Brightside is more similar to One Last Kiss, but the Cosine Similarity says that they are pointing in almost exactly the opposite directions (and are not similar at all). So...

- How do we know which to use?
- Is this enough?

Basic Ideas

Calculating the similarity score is basically just the **first** step in the basic **Collaborative Filtering Algorithm**, which is the focus of this lecture. The two most commonly used similarity scores are:

- The L2-norm:

$$||x - y||_2 = d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

- Generally used for lower dimensional settings because it is convenient (essentially can reduce to a k -nearest neighbors algorithm)
- The Cosine Similarity:

$$S_C(x, y) = \frac{x^T y}{\sqrt{x^T x} \sqrt{y^T y}}$$

- Most common because it scales well, and is popular in NLP settings with non-numeric data.

Depending on the context, you may use different distance measures, including L1-norm, L_∞ -norm, or others (e.g. the **Jaccard similarity** is commonly used for when data are made up of binary ratings (like/dislike)).

Today

- 1 Similarity
- 2 User-Based Filtering**
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Strictly Content-Based vs. Collaborative Filtering

The key idea behind collaborative filtering is that individuals with recorded **similar** preferences will continue to share **similar** preferences in the future:

- I like Mr. Brightside, and Mr. Brightside is most similar to One Last Kiss, so I will like One Last Kiss and continue to like other similar songs.
- Comparing the features of the songs (the items) to make recommendations is called **content-based filtering**.
 - You might notice that k -Nearest Neighbors is a type of content-based filtering.
- **Collaborative filtering** (either user based or item based; we'll talk about both) uses all the ratings of items by users to find similar users/items and aggregate them as predictions.
- There are **many** types of filtering algorithms, all with advantages and disadvantages. Often recommender systems use a **hybrid** approach, which combines several different types.
- We will start, however, with a very simple one: **User-user Collaborative Filtering**.

Basic Collaborative Filtering

There are generally 2 major steps:

- 1 Calculate similarities between a target user and other users based on their data/ratings of items (songs, movies, etc.) → identify similar users to the target user.
- 2 Predict ratings of items based on ratings of the items by the similar users → recommend an item (songs, movies, etc.) using the predicted ratings.

Basic Example

Consider five users and their ratings of four songs:

User	Song 1	Song 2	Song 3	Song 4
Dr. G	2		5	4
Student 1	5	3		4
Student 2	1	5	3	
Student 3		3	4	1
Student 4	3	4		4

One drawback to the user-user collaborative filtering approach is that because we usually have a very sparse (lot's of missing values; often users do not rate many items) data set, we need to impute missing values at their mean:

Basic Example

User	Song 1	Song 2	Song 3	Song 4
Dr. G	2	$\frac{11}{3}$	5	4
Student 1	5	3	4	4
Student 2	1	5	3	3
Student 3	$\frac{8}{3}$	3	4	1
Student 4	3	4	$\frac{11}{3}$	4

We then often **normalize (center and scale)** the ratings to correct for each user's average rating. If we assume the distribution of all users is similar, we can simplify our lives by simply **centering**:

Basic Example

Consider five users and their ratings (centered) of four songs:

User	Song 1	Song 2	Song 3	Song 4
Dr. G	-5/3	0	4/3	1/3
Student 1	1	-1	0	0
Student 2	-2	2	0	0
Student 3	0	1/3	4/3	-5/3
Student 4	-2/3	1/3	0	1/3

- **Note:** you can scale too if you like (as we did with the popularity and duration metrics).
- Also **Note:** when we fill in missing values with 0 now, we are saying that those are treated as "average" until we predict them later.
 - None of the students rated all the same songs Dr. Gerber rated; this strategy allows us to compare Dr. Gerber to **all** students (even if it makes some assumptions in order to do this)
 - There are definitely drawbacks to this approach, but it is a good place to start.

Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity**
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Finding Similar Users with Cosine Similarity

We can display the S_C scores (or other similarity scores) of the users now as a matrix/table, where each user is obviously exactly similar ($S_C = 1$) to themselves, where numbers nearer to -1 indicate the users have exactly opposite opinions/ratings/data:

	Dr. G	Student 1	Student 2	Student 3	Student 4
Dr. G	1	-0.55	0.55	0.26	0.69
Student 1	-0.55	1	-1	-0.11	-0.87
Student 2	0.55	-1	1	0.11	0.87
Student 3	0.26	-0.11	0.11	1	-0.25
Student 4	0.69	-0.87	0.87	-0.25	1

- An example of calculating one of these. From the previous slide we saw:

$$\bullet \text{ Dr. G} = \begin{bmatrix} -5/3 \\ 0 \\ 4/3 \\ 1/3 \end{bmatrix}$$

$$\bullet \text{ Student 1} = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

- So $S_C(\text{Dr. G}, \text{Student 1}) = -0.55$

Ranking Similarity

Given the S_C scores below, which user(s) are most similar to Dr. Gerber?

	Dr. G	Student 1	Student 2	Student 3	Student 4
Dr. G	1	-0.55	0.55	0.26	0.69
Student 1	-0.55	1	-1	-0.11	-0.87
Student 2	0.55	-1	1	0.11	0.87
Student 3	0.26	-0.11	0.11	1	-0.25
Student 4	0.69	-0.87	0.87	-0.25	1

- Student 4 and Student 2 are most similar to Dr. Gerber, Student 1 is least similar.
- How would we use Student 4 and Student 2 to predict what Dr. Gerber would think of the song he didn't listen to? Song 2?
 - Return to the original table:

Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations**
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering

Making Recommendations (incorrectly)

User	Song 1	Song 2	Song 3	Song 4
Dr. G	2	4.5	5	4
Student 1	5	3		4
Student 2	1	5	3	
Student 3		3	4	1
Student 4	3	4		4

- You might be inclined to say, let's just average the rating among the similar users!
 - Since Student 4 and Student 2 were most similar to Dr. Gerber, and their ratings for Song 2 were 4 and 5 (respectively), we could predict:

$$\text{Dr. G's Pred Song 2 Rating} = \frac{4 + 5}{2} = 4.5$$

Making Recommendations (incorrectly)

User	Song 1	Song 2	Song 3	Song 4
Dr. G	2	4.56	5	4
Student 1	5	3		4
Student 2	1	5	3	
Student 3		3	4	1
Student 4	3	4		4

- Instead, it is more common/precise to use the **weighted average** based on **the similarity scores**:

$$\text{Dr. G's Pred Song 2 Rating} = \frac{(0.69)4 + (0.55)5}{(0.69 + 0.55)} = 4.56$$

- **Caution:** even this isn't quite right, however...

Some Added Nuance

Notice that some of the similarity scores are negative:

	Dr. G	Student 1	Student 2	Student 3	Student 4
Dr. G	1	-0.55	0.55	0.26	0.69
Student 1	-0.55	1	-1	-0.11	-0.87
Student 2	0.55	-1	1	0.11	0.87
Student 3	0.26	-0.11	0.11	1	-0.25
Student 4	0.69	-0.87	0.87	-0.25	1

- What if you wanted to predict a missing song of Student 1? According to our S_C values, none of the students (or Dr. G) are similar to Student 1 and the weighted average we just calculated would not make sense (since lower magnitude negative values would be considered MORE similar when they shouldn't)
- To address this, we **scale** the similarity scores with Min-Max scaling, so that the lowest similarity score for a user is 0 and the highest is 1:

Some Added Nuance

Min-Max Scale Student 1's similarity scores (exclude themselves):

	Dr. G	Student 2	Student 3	Student 4
Student 1 (S_C)	-0.55	-1	-0.11	-0.87
Student 1 (MM)	0.510	0	1	0.150

- Notice that now, even if we used all the other users, the least similar user would not be considered in the predicted rating (weight of 0).
- Since Student 3 and Dr. G are the most similar to Student 1, we can use their ratings to predict Student 1's missing song (Song 3):

$$r(\text{St1}, \text{Song3}) = \frac{(1)4 + (.510)5}{1 + .510} = 4.34$$

- Note: If we do this (and we should) with Dr. G's Song 2 prediction, we would get (see **collab_filter.ipynb** for math):

$$r_{k=2}(\text{DrG}, \text{Song2}) = 4.47 \text{ or } r_{k=4}(\text{DrG}, \text{Song2}) = 4.09$$

Min-Max Scale with L2 norm

- How would we do the same thing with **Euclidean distance**? Remember that an L2 norm of 0 means exactly similar, while larger numbers are less similar. We can simply Min-Max scale the **negative L2 norm**:

$$-\infty < -L2 \leq 0 \rightarrow 0 < \frac{-L2 - \min(-L2)}{\max(-L2) - \min(-L2)} \leq 1$$

What if a similar user has **ALSO** not rated a item?

For the purposes of making sure we can always use similar user's data when predicting a rating for an item, we simply use the user's average rating. Notationally, if a predicted rating for user i and item j is given the k most similar users:

$$r(U_i, I_j) = \frac{\sum_k r(U_k, I_j) S_C(U_i, U_k)}{\sum_k S_C(U_i, U_k)}$$

If one of $r(U_k, I_j)$ is missing, we replace it with $\overline{r(U_k)}$, that similar user's average rating.

Overview of User-User CF

Given a set of ratings from users $U_i, i = 1, \dots$ on items $I_j, j = 1, \dots$:

- 1 Center (or standardize) all users given their mean rating
- 2 Impute missing ratings at 0 (or, at the user mean rating)
- 3 Compute pairwise similarity scores for users
- 4 Min-Max scale the similarity scores for each target user
- 5 Choose k to be the threshold for the most similar users
- 6 For a target user U_i and item I_j , calculate the predicted rating:

$$r(U_i, I_j) = \frac{\sum_k r(U_k, I_j) S_C(U_i, U_k)}{\sum_k S_C(U_i, U_k)}$$

- If one of $r(U_k, I_j)$ is missing, replace it with $\overline{r(U_k)}$, that similar user's average rating.

Practice

Let's practice with some real data. Below are a few famous movies which Dr. Gerber has seen, and a couple he hasn't. I'll randomly call out five students to provide their ratings, then we'll use collaborative filtering to find the two most similar students to determine what Dr. Gerber would rate the movies he hasn't seen:

User	Avatar	Frozen	Mean Girls	Star Wars	Titanic
Dr. G					
Student 1					
Student 2					
Student 3					
Student 4					
Student 5					

Drawbacks

Collaborative Filtering approaches often suffer from three problems:

- Cold start: for a new user or item, there is not enough data to make accurate recommendations
- Scalability: if there are millions of users and items, the act of calculating so many similarity scores can be computationally intensive
- Sparsity: most users only rate a small subset of the overall item set; even the most popular items will sometimes have very few ratings

Extensions

There are some extensions for the usage of basic CF, and some ways we can improve our recommendation system:

- Implement a hybrid of CF with **content based filtering**. One example (there are many different strategies):
 - Perform Step 1 of the CF algorithm to identify the most similar users to a target user.
 - Then, given a subset of items (e.g. highest rated songs) specific to those users and the target user, use clustering or similarity scores to recommend items that the target user has not rated.
- Item-item collaborative filtering (popularized by Amazon; we'll see an example shortly) can help address some of the scalability and sparsity issues of user-user filtering.
- Model-based filtering (using matrix decomposition approaches) is a slightly more advanced approach. I'll cover this if there is time, but if not you can see some basic examples at the end of the slides.
- **Note again:** User-user collaborative filtering with the L2-norm and a specific choice of k most similar users reduces to the k -Nearest Neighbors algorithm.

Including Content Based Filtering

In our original example, we've predicted that Dr. Gerber would rate Song 2 at 4.47 (so, he'll probably like it!), but what if we wanted to use more information about the songs he likes?

- Step 1 proceeds the same way, using ratings, we identify that Dr. Gerber is most like this subset of students (e.g. Student 4 and Student 2)
- Instead of directly predicting the rating of the songs Dr. Gerber hasn't listened to, collect all the songs rated highly by the similar students (say, all songs they've rated 4+) into a data set with measurements specific to those songs.
- Allow Dr. Gerber (the target user) to pick a song he likes (perhaps Mr. Brightside).
- Add Mr. Brightside to the data set, then either:
 - Perform clustering (perhaps k -means) and recommend song(s) from the data set that is/are in the same cluster as Mr. Brightside
 - Calculate the similarity scores for the songs based on their measurements and suggest the one(s) most similar to Mr. Brightside

Example

Suppose we've already completed Step 1 and found that everyone in DS 4420 is a similar user to Dr. Gerber, so we put their favorite songs into a data set. Based on some features (perhaps artist popularity, duration, if the song is explicit or not), and the fact that Dr. Gerber often listens to Mr. Brightside on repeat, which song(s) would we recommend to him?

Check out **collab_filter.ipynb** and **collab_filter.R** to see us do this.

Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering**
- 6 (if time) SVD-Based Filtering

Item-Item Collaborative Filtering

Invented and used by Amazon in 1998:

- matches each of the user's purchased and rated items to similar items
 - instead of calculating the pairwise user similarity scores, calculate pairwise **item** similarity scores
- combines those items into a recommendation list
 - use the items **already rated by the user** most similar to the item a user has not rated to predict the rating

Item-Item Collaborative Filtering

Item-item collaborative filtering has some benefits:

- it scales much better than user-user, since the number of users is often quite a bit larger than the number of items
- it handles sparsity much better than user-user, since it will essentially ignore any users who did not rate an item
 - What this means: it only compares ratings of users who have interacted with **both** items being compared.

Example

Here's the same table as before, but now we will **center on item**, and let's not worry about plugging in 0's for the missing ratings:

User	Song 1	Song 2	Song 3	Song 4
Dr. G	2		5	4
Student 1	5	3		4
Student 2	1	5	3	
Student 3		3	4	1
Student 4	3	4		4

User	Song 1	Song 2	Song 3	Song 4
Dr. G	-.75	-	1	.75
Student 1	2.25	-.75	-	.75
Student 2	-1.75	1.25	-1	-
Student 3	-	-.75	0	-2.25
Student 4	.25	.25	-	.75

Example

We are going to try to find the missing rating for Dr. G. To do this using item-item filtering, notice that we have 6 pairs of items to find similarity scores for:

- Song 1, Song 2
- Song 1, Song 3
- Song 1, Song 4
- Song 2, Song 3
- Song 2, Song 4
- Song 3, Song 4

Notice: Even if we had 1000 users, we would still only need to calculate these six similarity scores. Since there are generally many more users than items, this makes item-item CF a more scalable algorithm.

Example

The trick here is that we now only use the ratings of the users who have rated the Songs in the similarity calculation. For example, for $S_C(\text{Song 1}, \text{Song 2})$, only students 1, 2, and 4 rated both of these songs, so those are the only ratings we use:

User	Song 1	Song 2	Song 3	Song 4
Dr. G	-.75	-	1	.75
Student 1	2.25	-.75	-	.75
Student 2	-1.75	1.25	-1	-
Student 3	-	-.75	0	-2.25
Student 4	.25	.25	-	.75

$$\bullet \text{ Song 1} = \begin{bmatrix} 2.25 \\ -1.75 \\ .25 \end{bmatrix}$$

$$\bullet \text{ Song 2} = \begin{bmatrix} -.75 \\ 1.25 \\ .25 \end{bmatrix}$$

$$\bullet S_C = -0.9009$$

Example

For $S_C(\text{Song 1}, \text{Song 3})$, the vectors are of different length, since only Dr. G and Student 2 rated both of these songs:

User	Song 1	Song 2	Song 3	Song 4
Dr. G	-.75	-	1	.75
Student 1	2.25	-.75	-	.75
Student 2	-1.75	1.25	-1	-
Student 3	-	-.75	0	-2.25
Student 4	.25	.25	-	.75

- $\text{Song 1} = \begin{bmatrix} -.75 \\ -1.75 \end{bmatrix}$
- $\text{Song 3} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$
- $S_C = 0.3714$

Example

After doing it for all six pairs of songs (items, e.g. Song j is I_j), we get:

• $S_C(I_1, I_2) =$ -0.9009	• $S_C(I_1, I_4) =$ 0.4237	• $S_C(I_2, I_4) =$ 0.4842
• $S_C(I_1, I_3) =$ 0.3714	• $S_C(I_2, I_3) =$ -0.8575	• $S_C(I_3, I_4) =$ 0.3162

If we want to predict Dr. G's Song 2 rating, we Min-Max scale and take the weighted average of Dr. G's ratings for the k most similar songs:

Example

Min-Max Scaled Song 2 scores:

$$S_C(I_1, I_2) = 0$$

$$S_C(I_2, I_3) = 0.031$$

$$S_C(I_2, I_4) = 1$$

- **Note:** the predicted rating of an item j for a user i ($r(U_i, I_j)$) is the weighted average of that user's ratings of items similar to item j .

Dr. G's Pred Song 2 Rating =

$$\frac{r(U_{DrG}, I_1)S_C(I_1, I_2) + r(U_{DrG}, I_3)S_C(I_2, I_3) + r(U_{DrG}, I_4)S_C(I_2, I_4)}{S_C(I_1, I_2) + S_C(I_2, I_3) + S_C(I_2, I_4)} =$$
$$\frac{2 * 0 + 5 * 0.031 + 4 * 1}{0 + 0.031 + 1} = \frac{4.16}{1.031} = 4.03$$

Overview of Item-Item CF

Given a set of ratings from users $U_i, i = 1, \dots$ on items $I_j, j = 1, \dots$:

- 1 Center (or standardize) all items given their mean rating
- 2 Compute pairwise similarity scores for items using only the users who have rated both items
- 3 Min-Max scale the similarity scores for each target item
- 4 Choose k to be the threshold for the most similar items
- 5 For a target user U_i and item I_j , calculate the predicted rating:

$$r(U_i, I_j) = \frac{\sum_k r(U_i, I_k) S_C(I_j, I_k)}{\sum_k S_C(I_j, I_k)}$$

- If one of $r(U_i, I_k)$ is missing, replace it with $\overline{r(I_k)}$, that similar item's average rating.

Today

- 1 Similarity
- 2 User-Based Filtering
- 3 Step 1: Similarity
- 4 Step 2: Recommendations
- 5 Item-Based Filtering
- 6 (if time) SVD-Based Filtering**

Matrix Factorization Method

- User- and Item-based methods rely on direct neighbor comparisons (using similarity scores), but struggle with sparsity and computation
- Matrix Factorization can be used to solve these issues, handling larger data matrices more efficiently
- It still does not solve the cold start issue though... content-based filtering (hybrid approach) can still be helpful

Matrix Factorization Method

Matrix factorization gives a way to learn user and item relationships in a more computationally efficient way than the neighborhood approaches we've already seen.

- You may have done **eigen decomposition** of matrices in ML1 for PCA and LDA purposes.
- The most common approach for collaborative filtering is based on **singular value decomposition**
 - As with the “neighborhood” approaches (user- and item-based) there are many different ways to approach this. We'll talk about one.

Singular Value Decomposition

To factorize the matrix of ratings (let's call it R for ratings), we want to find the matrices U , S , and V such that:

$$R = USV^T$$
$$\begin{bmatrix} R_{11} & \cdots & R_{1d} \\ \vdots & \ddots & \vdots \\ R_{n1} & \cdots & R_{nd} \end{bmatrix} = \begin{bmatrix} U_{11} & \cdots & U_{1r} \\ \vdots & \ddots & \vdots \\ U_{n1} & \cdots & U_{nr} \end{bmatrix} \begin{bmatrix} S_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & S_{rr} \end{bmatrix} \begin{bmatrix} V_{11} & \cdots & V_{d1} \\ \vdots & \ddots & \vdots \\ V_{1r} & \cdots & V_{dr} \end{bmatrix}$$

- where n is the number of users and d the number of items
- **Note:** every matrix R can be written as $R = USV^T$, where $U^T U = I$, $V^T V = I$, and S is diagonal with $S_{ii} \geq 0$
- $r = \text{rank}(R)$, the maximum number of linearly independent rows or columns it has

SVD for Recommender Systems

We will skip (for time's sake) some of the underlying math. How to interpret the matrix decomposition? Why does this help with recommender systems?

- Note that if we took RR^T , that would create a matrix of user-to-user ratings (each element (i, j) is the dot product of the i -th and j -th user's ratings, showing how similar their ratings are; you may also note this is the numerator of the cosine similarity...)
- Correspondingly, $R^T R$ is the item-item matrix
- The SVD matrices U and V are made up of the **eigenvectors** of those matrices! This means they capture latent structure/hidden relationships

SVD for Recommender Systems

We will skip (for time's sake) some of the underlying math. How to interpret the matrix decomposition? Why does this help with recommender systems?

- The SVD matrices U and V are made up of the **eigenvectors** of the user-user and item-item matrices! This means they capture latent structure/hidden relationships
- To use them to predict user-item ratings, we **truncate** them to keep the $k < r$ “most important” latent factors: $\tilde{U} \in \mathbb{R}^{n \times k}$, $\tilde{V}^T \in \mathbb{R}^{k \times d}$, and $\tilde{S} \in \mathbb{R}^{k \times k}$
- Then, reconstruct $\hat{R} = \tilde{U}\tilde{S}\tilde{V}^T$ to get the predictions

Final Notes

- We once again need to **center** and **impute** missing values at 0 in order to compute the SVD of the user-item matrix
 - Whether you use the user means or item means to center results in different strategies:
 - User means: forces the model to learn what a user likes relative to their own average behavior.
 - Item means: forces the model to learn which users have similar tastes in quality.
 - You can always do both and then average them for a “hybrid” approach.
- This approach is the foundation for the more complex **Funk Matrix Factorization**, which is **model**-based. It optimizes the number of latent factors (and the contribution of U and V to the predictions) by minimizing the objective function:

$$\mathcal{L} = ||R - \hat{R}||_F + \alpha ||U|| + \beta ||V^T||$$

- where $|| \cdot ||_F$ is the **frobenius norm**, and the norms on U and V can be any norm (like the l2)
- Even if I don't have time to go through this approach, it should be reasonable to implement (I'm pretty sure Funk gives some basic code in that blog post)

In Python and R

Follow along with a simple implementation (SVD) in **collab_filter.ipynb** and **collab_filter.R**.